Full-Stack Developer Intern

Technical Assessment

GlobalTNA

---

1. Overview

Thanks for your interest in the Full-Stack Developer Intern role at GlobalTNA. This is assessment structure code, and use the stack we work with day-to-day.

The scope is intentionally small. We are not looking for a finished product. We want a working slice of a full-stack app and a clear README. Please do not over-engineer it.

2. Required Tech Stack

- **Frontend:** Next.js (App Router preferred, Pages Router accepted)
- **Backend:** Node.js + Express (separate from the Next.js app)
- **Database:** MongoDB (Atlas free tier or local install)
- **ODM:** Mongoose recommended, not mandatory
- **Styling:** Anything you like - Tailwind, CSS modules, plain CSS

3. The Brief - Mini Service Request Board

Build a small web app where a homeowner can post a service request (for example, "Need a plumber for a leaking kitchen tap in Glasgow") and tradespeople can browse open requests, view details, and mark them as in progress or closed.

Think of it as a stripped-down, single-page version of the platform we are building.

3.1 Data Model - JobRequest

Create a single MongoDB collection called jobRequests with the following fields:

- **title** - string, required
- **description** - string, required
- **category** - string (e.g. Plumbing, Electrical, Painting, Joinery)
- **location** - string (e.g. Glasgow)
- **contactName** - string
- **contactEmail** - string, validate email format
- **status** - enum: "Open" | "In Progress" | "Closed", default "Open"
- **createdAt** - Date, auto-set on create

3.2 Backend API (Express)

Build a REST API. Use proper HTTP status codes and return JSON.

- **GET /api/jobs** - list all jobs; support optional filters ?category=Plumbing and ?status=Open
- **GET /api/jobs/:id** - fetch a single job
- **POST /api/jobs** - create a new job (validate required fields)
- **PATCH /api/jobs/:id** - update status only
- **DELETE /api/jobs/:id** - delete a job

Include basic input validation and a global error handler. A 404 for missing resources should be obvious.

3.3 Frontend (Next.js)

Keep the UI clean and functional. Three screens are enough:

1. **Home page** - list of all job requests as cards or a table, with a category filter dropdown.
2. **New job form** - form to create a new request, with client-side validation.
3. **Job detail page** - full details, dropdown to change status, delete button.

The frontend must talk to your Express API - not directly to MongoDB or via Next.js API routes only.

4. Bonus (Optional)

Only attempt these if the core brief is solid.

- Keyword search across title and description.
- JWT-based auth so only logged-in users can post or delete.
- Deploy frontend to Vercel and backend to Render or Railway, share live URLs.
- Unit tests on at least one or two API endpoints (Jest or Vitest).
- Simple seed script to insert 5-10 sample jobs.

.

Submission Details

- Complete and submit the assignment by 18 May 2026 (Mon) at 12:00 pm.
- Push the completed code to a public GitHub repository.
- Include a clear README.md file in the repository root with:
    - Setup instructions
    - Required environment variables
    - Run instructions for both frontend and backend
- Email the GitHub repository and demo link to your point of contact at GlobalTNA.
- CC: nimeshsago@gmail.com in the submission email.

Good luck - we are looking forward to seeing what you build.